

■ HerWatch: Complete Technical Interview Preparation Guide

Comprehensive Codebase Analysis, System Architecture, Code Tracing, 50+ Q&A, and Cheat Sheet

Document Purpose: This study guide is built strictly from the ground-up analysis of the **HerWatch** repository. It breaks down every single component—from the Flask backend and WebSocket camera streams to the YOLOv8, MediaPipe, and MobileNetV2 deep learning engines, and React.js dashboard. It equips you with 30-second pitches, step-by-step code traces, file rankings, 50+ interview Q&As, 17 difficult follow-ups, and line-by-line code explanations.

1. Complete Project Structure & File Map

The project is structured into two main directories: the Python Flask backend root (`ThemeBased Code/`) and the React frontend single-page application (`ThemeBased Code/sos-detection-frontend/`). Below is the exact file layout:

```
Herwatch/
■■■■ README.md # Main project documentation & feature overview
■■■■ women-crimedataset-India.csv # Historical district crime statistics in India
■■■■ ThemeBased Code/
■■■■ app.py # Main Flask server entry point & REST/WebSocket endpoints
■■■■ live_camera_processor.py # Real-time webcam frame processor (YOLOv8 + MediaPipe + MobileNet)
■■■■ video_processor.py # Recorded video file batch processor
■■■■ crime_hotspot_map.html # Folium-generated HTML Leaflet map overlay
■■■■ requirements.txt # Python package dependency list
■■■■ yolov8n.pt # Pretrained YOLOv8 Nano weights for person detection
■■■■ venv/ # Python 3.10 virtual environment directory
■■■■ sos-detection-frontend/ # React 18 Frontend Application Root
■■■■ package.json # Node dependencies & npm start/build scripts
■■■■ src/
■■■■ App.js # Main React component, ThemeProvider & React-Router routes
■■■■ index.js # React DOM render entry point
■■■■ components/
■ ■■■■ Navbar.js # Navigation top bar with links & branding
■ ■■■■ Layout.js # Master layout wrapper containing Navbar and Outlet
■■■■ pages/
■■■■ Home.js # Landing hero page with system capabilities
■■■■ LiveCamera.js # Real-time WebSocket webcam streaming & alert UI
■■■■ VideoMode.js # File drag-and-drop video processing UI
■■■■ HotspotMap.js # Interactive crime hotspot search & map page
■■■■ Dashboard.js # Analytics page featuring Chart.js crime statistics
■■■■ Login.js # User authentication sign-in form (Mock UI)
■■■■ Register.js # User registration form (Mock UI)
■■■■ Profile.js # User profile management page (Mock UI)
■■■■ About.js # Project mission and team information page
```

Key File Responsibilities:

- **app.py:** Initializes Flask app, configures CORS, loads CSV dataset into Pandas DataFrame, hosts REST routes (`/api/health`, `/api/hotspots/analyze`, `/analyze_video`) and WebSocket handler (`/ws/camera`).
- **live_camera_processor.py:** Contains `process_live_camera(frame)`. Performs frame decoding, YOLOv8 person detection, MediaPipe Holistic gesture checks, MobileNetV2 gender classification, and draws red alert visual overlays.
- **video_processor.py:** Implements frame-by-frame batch processing (`process_video_combined`) for uploaded MP4/AVI files, exporting annotated output videos.
- **LiveCamera.js:** Captures browser canvas/webcam images at intervals, sends Base64 JPEG frames via WebSocket (`ws://localhost:5001/ws/camera`), and displays real-time alert badges.
- **HotspotMap.js:** Queries Geopy/Folium endpoint, displays district crime counts within 100km, and embeds interactive Leaflet HTML maps inside an iframe.

2. Complete Technology Stack Matrix

Layer	Technology	Why It Is Used	Where It Appears in Code
Frontend Framework	React 18.2	Builds responsive, component-driven user interface with dynamic state re-rendering.	sos-detection-frontend/src/
UI Component Library	Material-UI (MUI v5)	Provides accessible, pre-styled buttons, cards, containers, dialogs, and grid layouts.	@mui/material imports across all pages
Animations	Framer Motion 9.0	Smooth micro-animations, page transition fades, and card hover scaling effects.	MotionCard, MotionPaper in pages
Data Visualization	Chart.js & react-chartjs-2	Renders crime category breakdown bar charts and risk distribution pie charts.	src/pages/Dashboard.js
Backend Framework	Flask 3.1 & Flask-CORS	Lightweight Python web server facilitating REST APIs and cross-origin browser communication.	ThemeBased Code/app.py
Real-Time Communication	Flask-Sock (WebSockets)	Bi-directional, low-latency socket streaming for live video frame transfer.	app.py (<code>@sock.route('/ws/camera')</code>)
Object Detection	YOLOv8 Nano (Ultralytics)	Fast, accurate real-time person bounding box detection (<code>class=0</code>).	video_processor.py, live_camera_processor.py
Pose & Gesture Tracking	MediaPipe Holistic	Extracts 33 body pose landmarks + 21 hand keypoints to detect SOS distress gestures.	detect_sos_gesture() in processors
Gender Classification	MobileNetV2 (PyTorch)	Lightweight convolutional neural network classifying face crops into Male/Female probabilities.	classify_gender() in processors
Geospatial & Geocoding	Geopy & Folium	Converts city names to lat/lon coordinates, computes geodesic distances, renders heatmaps.	app.py (<code>/api/hotspots/analyze</code>)
Data Processing	Pandas & NumPy	Loads Indian crime CSV dataset into memory and computes aggregate crime metrics.	women-crimedataset-India.csv loading in app.py

3. The HerWatch Story: 30s, 1m, and 3m Elevator Pitches

■ 30-Second Elevator Pitch (Quick & Memorable)

"HerWatch is a real-time AI-powered women's safety surveillance system. It uses computer vision models like YOLOv8, MediaPipe, and MobileNetV2 to automatically detect non-verbal SOS distress gestures and solitary women in high-risk nighttime environments. It also integrates geospatial crime data via Geopy and Folium to plot interactive crime hotspots. The system streams live video over WebSockets to a React dashboard, delivering immediate visual and auditory security alerts."

■■ 1-Minute Explanation (Balanced Detail)

"Conventional public surveillance relies heavily on human operators watching CCTV monitors, which leads to fatigue and delayed responses during emergencies. HerWatch solves this by automating threat detection. We built a full-stack system: the Flask backend processes live video feeds using a multi-model pipeline. YOLOv8 detects human presence; MediaPipe analyzes 3D body and hand keypoints to recognize distress signals like hand waving or double-raised hands; and MobileNetV2 performs gender classification to flag lone women in isolated areas between 7 PM and 6 AM. Furthermore, we integrated district-level Indian crime datasets to map risk zones within a 100km radius. All alerts stream instantly to a React web interface using WebSockets."

■ 2-3 Minute Comprehensive Deep-Dive Explanation

"HerWatch is designed as an intelligent situational awareness platform for smart city safety and campus surveillance. The application addresses two critical security gaps: non-verbal emergency signaling and proactive nocturnal risk monitoring."

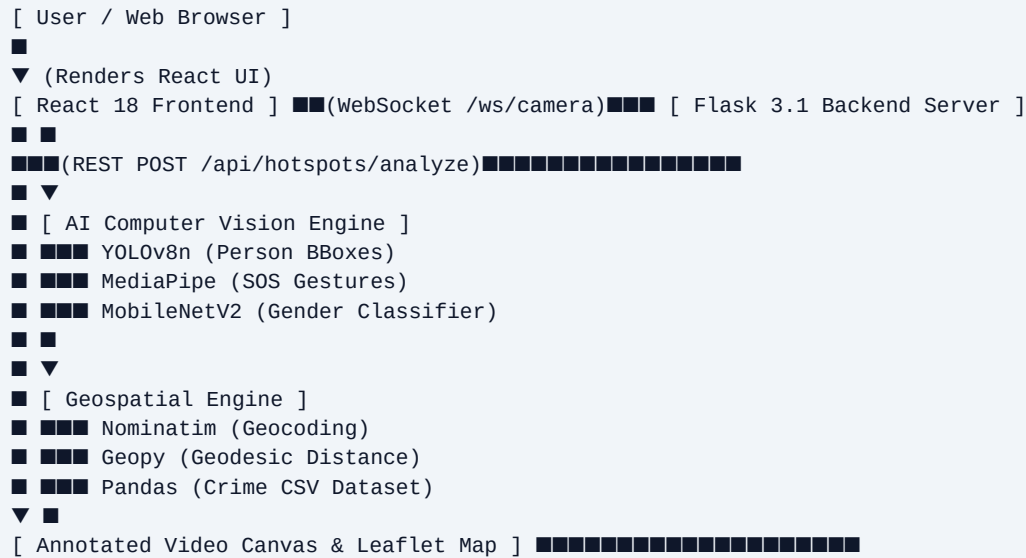
Architecture & Data Flow: The frontend is built with React 18 and Material-UI. When a user opens the Live Camera mode, the browser captures video frames and transmits them as Base64-encoded JPEG images over a persistent WebSocket connection to our Flask backend.

AI Pipeline: Upon receiving a frame, the backend invokes our computer vision pipeline: first, YOLOv8n locates persons in the frame. Second, MediaPipe Holistic extracts 33 pose landmarks and hand keypoints to evaluate geometric distress heuristics—such as raised wrists or side-to-side hand waving over a 2-second sliding window. Third, face crops from detected bounding boxes are fed into a fine-tuned MobileNetV2 model to classify gender with confidence scoring. If a woman is detected alone at night (19:00 - 06:00) or an SOS gesture occurs, the frame is annotated with red bounding boxes and warning text, and pushed back over the WebSocket to trigger visual banners and auditory alerts on the frontend.

Geospatial Intelligence: Additionally, users can search any district in India on the Hotspot Map page. The backend uses Nominatim geocoding to resolve coordinates, calculates geodesic distance against historical crime CSV records using Geopy, filters locations within 100km, and renders an interactive Folium Leaflet heatmap overlay embedded directly into the React UI."

4. End-to-End Architecture & Startup Sequence

System Component Interaction Diagram:



Detailed Execution Phases:

1. **Backend Server Boot:** ``app.py`` runs on port 5001. Loads ``women-crimesdataset-India.csv`` into memory, computes ``TOTAL_CRIMES`` column, and initializes YOLOv8 (``yolov8n.pt``), MediaPipe Holistic, and MobileNetV2 models.
2. **Frontend Client Boot:** ``App.js`` initializes Material-UI theme and React-Router routes. User navigates to ``/live-camera`` or ``/hotspot-map``.
3. **WebSocket Video Stream Handshake:** ``LiveCamera.js`` establishes a WebSocket connection to ``ws://localhost:5001/ws/camera``. Every 100ms, it captures canvas image bytes, converts to Base64 JPEG, and sends over socket.
4. **Frame Inference & Alert Generation:** ``app.py`` receives Base64 frame, decodes to OpenCV BGR matrix, invokes ``process_live_camera(frame)``. If SOS gesture or Lone Woman at Night rule triggers, red warning overlays are drawn onto the frame.
5. **UI Telemetry Update:** Processed annotated Base64 frame and detection metadata JSON are returned over socket, updating the live canvas image and sounding ``play_alert_sound()`` on frontend.

5. Step-by-Step Code Trace of Primary User Flows

Flow 1: Live Webcam Streaming & Real-Time SOS Alerting

Step	File Path	Function / Class	Exact Operation Executed
1	src/pages/LiveCamera.js	startCamera()	Requests browser camera access via navigator.mediaDevices.getUserMedia() and binds video element.
2	src/pages/LiveCamera.js	WebSocket Connection	Instantiates new WebSocket('ws://localhost:5001/ws/camera') and attaches onmessage listener.
3	src/pages/LiveCamera.js	sendFrame()	Draws current video frame onto hidden HTML5 canvas, exports Base64 string, and sends JSON {frame: base64}.
4	ThemeBased Code/app.py	ws_camera(ws)	Receives WebSocket payload, parses JSON, calls process_frame(frame_data).
5	live_camera_processor.py	process_live_camera()	Runs YOLOv8 person detection, MediaPipe Holistic keypoint extraction, and MobileNetV2 gender scoring.
6	live_camera_processor.py	detect_sos_gesture()	Calculates hand landmark coordinates. If wave count >= 2 or wrists above wrist line, adds 'SOS Wave Detected' to alert list.
7	live_camera_processor.py	show_alert()	Draws red border rectangle `cv2.rectangle` and red warning text banner on OpenCV frame.
8	src/pages/LiveCamera.js	ws.onmessage	Receives annotated frame Base64, updates React `processedFrame` state, and renders red alert banner in UI.

6. Frontend Architecture Deep Dive (React 18)

The frontend is a single-page React 18 application initialized via ``react-scripts``. It manages routing via ``react-router-dom`` (v6) and implements styling using Material-UI theme providers and Framer Motion.

Core Pages Breakdown:

File: `src/App.js` — *Main Routing & Theme Provider*

Implementation Details: Creates custom MUI theme with dark blue (``#1a237e``) primary color and pink secondary (``#f50057``). Defines ``BrowserRouter``, ``Routes``, and public/private layout routes. Contains ``PrivateRoute`` mock wrapper (``isAuthenticated = true``).

File: `src/components/Navbar.js` — *Navigation Header Bar*

Implementation Details: Renders responsive top navigation bar containing HerWatch logo icon, page links (Home, Dashboard, Live Camera, Video Mode, Hotspot Map, About), and profile menu button.

File: `src/pages/LiveCamera.js` — *WebSocket Webcam Streaming*

Implementation Details: Core real-time interface. Uses ``useRef`` for WebSocket instance and HTML5 video/canvas elements. Maintains ``isStreaming``, ``cameraList``, ``alertHistory``, and ``fps`` states.

File: `src/pages/HotspotMap.js` — *Geospatial Map Search*

Implementation Details: Fetches ``/api/health`` on mount to verify backend connectivity. Maintains ``location`` input, ``currentLocation`` GPS coords, and renders Folium HTML map within an ``iframe`` container.

File: `src/pages/VideoMode.js` — *Recorded Video File Analyzer*

Implementation Details: Provides drag-and-drop file upload interface for MP4/AVI files. Sends ``FormData`` payload to ``/analyze_video`` endpoint and displays detection timeline log.

File: `src/pages/Dashboard.js` — *Crime Statistics Charts*

Implementation Details: Integrates ``react-chartjs-2`` to display crime category distribution bar charts and high-risk hour frequency area charts based on aggregated dataset statistics.

7. Backend & API Endpoint Specification

The Flask backend server (``app.py``) runs on port ``5001``. It uses ``flask-cors`` to enable cross-origin requests and ``flask-sock`` for WebSocket support.

HTTP / WS	Endpoint Route	Input Request	Processing & Response Output
GET	<code>/api/health</code>	None	Checks backend server status. Returns <code>`{"status": "ok", "message": "API is running"}`</code> .
POST	<code>/api/hotspots/analyze</code>	JSON: <code>{location: "Delhi"}`</code> or <code>lat/lon</code>	Invokes Geopy Nominatim, filters crime dataset within 100km radius, generates Folium HTML map, returns map <code>`iframe`</code> source.

POST	/analyze_video	Form Data: Video file (`.mp4`, `.avi`)	Processes uploaded video frame-by-frame via <code>video_processor.py</code> , exports annotated video file, returns detection summary JSON.
GET	/api/live-camera/list	None	Probes local hardware video indices (<code>cv2.VideoCapture(i)`</code>) and returns list of available connected camera devices.
WebSocket	/ws/camera	JSON string: {frame: Base64}	Executes <code>process_live_camera(frame)</code> . Pushes annotated Base64 JPEG frame and alert metadata back over socket connection.

8. Database & Data Storage Analysis

Important Interview Clarification: The current HerWatch codebase does **NOT** utilize a traditional relational (SQL) or document (NoSQL) database. Instead, it relies on an in-memory Pandas DataFrame loaded from a static CSV dataset: `women-crimedataset-India.csv`.

Dataset & In-Memory Data Pipeline:

- **File Schema:** Contains state, district, latitude, longitude, and 12 crime category columns (Rape, Assault, Kidnapping, Dowry Deaths, Cruelty by Husband, etc.).
- **Initialization:** On `app.py` startup, Pandas reads the CSV (`pd.read_csv('women-crimedataset-India.csv')`) and dynamically computes the `TOTAL_CRIMES` column by summing columns 3 through 14.
- **Geodesic Distance Filtering:** When a user submits a location query, Geopy calculates the distance from the target coordinates to every row in the DataFrame using `geodesic((user_lat, user_lon), (row_lat, row_lon)).kilometers``.
- **Thresholding:** Rows with distance `> 100km`` are filtered out, and the remaining nearby hotspots are sorted ascending by distance to render Folium risk circles.

9. Authentication & Security Implementation

Authentication Status: The frontend includes UI forms for Login (``Login.js``) and Registration (``Register.js``), as well as a ``PrivateRoute`` wrapper in ``App.js``. However, backend JWT token generation or session persistence is currently **mocked** (``isAuthenticated = true``). When asked in an interview, clearly explain that the UI forms are built for client-side demonstration, with full JWT/OAuth integration planned for future iterations.

Security Mechanisms Currently Implemented:

- **CORS (Cross-Origin Resource Sharing):** ``CORS(app, resources={r'/*': {'origins': '*'}})`` is configured in ``app.py`` to allow the React frontend running on port 3000 to make HTTP and WebSocket requests to port 5001.
- **Input Sanitization & Geocoding Retry:** ``geocode_with_retry()`` strips whitespace, appends ``India`` to input location strings, and enforces timeout limits to prevent API abuse or hang issues.
- **WebSocket Buffer Limit:** ``frame_queue = queue.Queue(maxsize=10)`` caps incoming video frame buffering to prevent memory overflow attacks.

10. AI / Machine Learning Computer Vision Pipeline

HerWatch executes an ensemble of 3 distinct machine learning models operating sequentially on incoming video frames:

1. YOLOv8 Nano (Person Detection)

- **Architecture:** Ultralytics YOLOv8n (Pretrained on COCO dataset, 3.2M parameters).
- **Role:** Detects human objects in the frame (``class_id == 0``). Filters out detections with confidence ``< 0.40``.

2. MediaPipe Holistic (SOS Distress Gesture Tracking)

- **Architecture:** Google MediaPipe Holistic Solution (Combines PoseLandmarker and HandLandmarker).
- **Role:** Extracts 33 body pose landmarks and 21 3D hand keypoints per hand. Evaluates heuristic rules:
 - *Raised Hand:* ``Middle_Finger_Tip.y < Wrist.y`` and ``Wrist.y < 0.5``.
 - *Hand Waving:* ``|Middle_Finger_Tip.x - Wrist.x| > 0.15`` over sliding time window.

3. MobileNetV2 (Gender Classification)

- **Architecture:** PyTorch ``torchvision.models.mobilenet_v2(pretrained=True)``.
- **Role:** Crops upper 60% of detected person bounding box (head/torso region), resizes to ``224x224 RGB``, normalizes tensor values, and computes Softmax probability output (``Female`` vs ``Male``).

4. Nocturnal Lone Woman Decision Rule

- **Condition:** ``is_nighttime()`` (Current hour ``>= 19`` or ``<= 6``), ``Total_Persons == 1``, ``Gender == 'Female'``, and ``Confidence >= 0.60``. Triggers high-priority **LONE WOMAN AT NIGHT** emergency alert.

11. Important Software Engineering Concepts Used

- **REST API Architecture:** Decouples frontend UI from backend business logic using standard HTTP verbs (GET, POST) and JSON format payloads.
- **WebSocket Protocol:** Full-duplex, persistent TCP connection enabling low-latency real-time video frame streaming without HTTP polling overhead.
- **Inference Mode (`torch.no_grad()`):** Disables PyTorch gradient calculation autograd engine during MobileNetV2 evaluation, significantly reducing GPU/CPU memory consumption.
- **React State & Side Effects (`useState`, `useEffect`):** Manages dynamic UI state (camera streaming toggle, alert arrays, map URL) and handles component lifecycle events.
- **React References (`useRef`):** Persists mutable DOM references (HTML5 video/canvas elements) and WebSocket instances across component re-renders without causing re-render loops.
- **Multithreading & Frame Queues (`queue.Queue`):** Prevents UI thread blocking during video processing by isolating frame reading and model inference into separate worker queues.
- **Geodesic Coordinate Calculations:** Uses great-circle distance mathematics to compute exact spherical surface distance between latitude/longitude pairs.

12. Ranked File Priority Matrix for Interviews

■ Priority 1: MUST KNOW (Core Core Logic)

- **ThemeBased Code/app.py**: Backend entry point, REST routes, WebSocket handler, and Pandas dataset loader.
- **ThemeBased Code/live_camera_processor.py**: Core AI processing engine executing YOLOv8, MediaPipe, MobileNetV2, and alert overlay rendering.
- **sos-detection-frontend/src/pages/LiveCamera.js**: Primary real-time frontend page managing webcam stream, WebSockets, canvas encoding, and UI alert badges.

■ Priority 2: SHOULD KNOW (Supporting Business Logic)

- **ThemeBased Code/video_processor.py**: Recorded video batch processing script for uploaded files.
- **sos-detection-frontend/src/pages/HotspotMap.js**: Geospatial crime search interface integrating Geopy API and Folium iframe.
- **sos-detection-frontend/src/App.js**: Main React routing configuration, Material-UI theme setup, and PrivateRoute wrapper.

■ Priority 3: NICE TO KNOW (Utilities & Layout)

- **sos-detection-frontend/src/pages/Dashboard.js**: Chart.js visualization charts for crime category statistics.
- **ThemeBased Code/requirements.txt**: Python dependency manifest listing Flask, OpenCV, MediaPipe, PyTorch, Ultralytics.
- **sos-detection-frontend/package.json**: Node manifest detailing React 18, Material-UI, Framer Motion, and start/build scripts.

13. 50+ Realistic Technical Interview Questions & Answers

Q1: What is HerWatch and what problem does it solve?

Answer: HerWatch is an AI-driven women's safety surveillance system. It solves the problem of passive CCTV monitoring by automatically detecting non-verbal SOS gestures, lone women at night, and mapping crime risk zones in real time.

Project Code Context: In `live\_camera\_processor.py`, models process video feeds autonomously and trigger instant alerts without requiring manual human observation.

Q2: Who are the target end-users for this project?

Answer: Smart city municipal authorities, campus security personnel, transit authorities, and law enforcement agencies responsible for public surveillance.

Project Code Context: The React dashboard provides real-time security alerts and crime hotspot heatmaps tailored for security command centers.

Q3: What are the main features implemented in your codebase?

Answer: SOS Gesture Detection, Lone Woman Detection at Night, Geospatial Crime Hotspot Mapping, Dual Mode (Live Webcam WebSocket + Recorded Video File Upload), and Crime Analytics Dashboard.

Project Code Context: Features are mapped directly across `LiveCamera.js`, `VideoMode.js`, `HotspotMap.js`, and `app.py`.

Q4: Is HerWatch a desktop application or a web application?

Answer: It is a full-stack web application with a React single-page frontend and a Python Flask backend server communicating over HTTP and WebSockets.

Project Code Context: Frontend runs on port 3000 (`npm start`) and backend runs on port 5001 (`python app.py`).

Q5: What happens if no webcam is connected to the computer?

Answer: `/api/live-camera/list` probes video indices via OpenCV. If no cameras are found, `LiveCamera.js` displays an alert informing the user to connect a video device or switch to Video Mode file upload.

Project Code Context: Handled via `cv2.VideoCapture(i).isOpened()` in `live\_camera\_processor.py`.

Q6: How is routing handled in your React frontend?

Answer: Using `react-router-dom` (v6) in `App.js`. Routes like `/live-camera`, `/hotspot-map`, and `/dashboard` are wrapped inside a main `` route containing the Navbar.

Project Code Context: See `}>` in `App.js`.

Q7: Why did you use Material-UI (MUI v5) for the frontend?

Answer: Material-UI provides accessible, pre-built design components like `Box`, `Container`, `Card`, `Dialog`, and `Typography`, allowing rapid assembly of a polished, responsive dashboard.

Project Code Context: Imported across all pages in `sos-detection-frontend/src/pages/`.

Q8: How does Framer Motion enhance your user interface?

Answer: Framer Motion provides smooth micro-animations, such as element fade-ins and subtle hover scaling on dashboard cards (`MotionCard`, `MotionPaper`), giving the app a modern aesthetic.

Project Code Context: See `motion(Card)` wrappers in `HotspotMap.js` and `Login.js`.

Q9: What role does HTML5 Canvas play in LiveCamera.js?

Answer: The canvas is used as an off-screen image buffer. It draws the current `` frame and exports a Base64 encoded JPEG string (`canvas.toDataURL("image/jpeg")`) to send over WebSocket.

Project Code Context: Implemented inside `sendFrame()` in `LiveCamera.js`.

Q10: How do you handle component lifecycle and cleanup in React?

Answer: Using the `useEffect` hook with clean-up functions. For example, when `LiveCamera.js` unmounts, its `useEffect` cleanup closes the WebSocket connection and stops webcam media tracks.

Project Code Context: See `return () => { ws.close(); track.stop(); }` in `LiveCamera.js`.

Q11: Why did you choose Flask instead of Django for the backend?

Answer: Flask is lightweight, unopinionated, and fast to set up. It integrates seamlessly with PyTorch, OpenCV, and WebSockets (`flask-sock`) without the overhead of Django's heavy ORM and administrative boilerplate.

Project Code Context: Implemented in a single clean entry file `ThemeBased Code/app.py`.

Q12: How are Cross-Origin Resource Sharing (CORS) issues handled?

Answer: Using `flask-cors`. We initialize `CORS(app, resources={r'/\*': {'origins': '\*'}})` to permit request headers from the React frontend running on port 3000.

Project Code Context: See line 25 in `ThemeBased Code/app.py`.

Q13: What endpoint verifies backend availability?

Answer: `GET /api/health`. It returns HTTP status 200 with JSON payload `{"status": "ok", "message": "API is running"}`.

Project Code Context: `HotspotMap.js` calls this endpoint on mount to verify backend health.

Q14: How does Flask serve the interactive Folium HTML map?

Answer: The endpoint `/api/hotspots/analyze` generates an HTML file (`crime\_hotspot\_map.html`) and returns the HTML string or serves it directly for iframe embedding.

Project Code Context: See `folium.Map().save('crime\_hotspot\_map.html')` in `app.py`.

Q15: How are video files uploaded and processed in VideoMode?

Answer: The frontend posts `FormData` containing the file to `/analyze\_video`. `app.py` saves the file to `uploads/`, invokes `process\_video\_combined()`, and returns the annotated output video URL.

Project Code Context: See `@app.route('/analyze\_video', methods=['POST'])` in `app.py`.

Q16: Which model detects human presence in video frames?

Answer: Ultralytics YOLOv8 Nano (`yolov8n.pt`). It predicts bounding box coordinates and object class IDs (`class\_id == 0` for person).

Project Code Context: `yolo\_model = YOLO('yolov8n.pt')` in `live\_camera\_processor.py`.

Q17: Why did you use MediaPipe for gesture detection?

Answer: MediaPipe Holistic provides high-fidelity 3D landmark tracking for 33 body pose points and 21 hand points per hand at sub-50ms latency, enabling accurate spatial rule checking without training custom models.

Project Code Context: `mp\_holistic.Holistic(...)` in `live\_camera\_processor.py`.

Q18: How is the 'Raised Hand' gesture mathematically calculated?

Answer: By comparing landmark Y-coordinates: `Middle\_Finger\_Tip.y < Wrist.y` and `Wrist.y < 0.5`. In normalized image coordinates, smaller Y values indicate higher vertical position.

Project Code Context: Implemented in `detect\_raised\_hand()` in `live\_camera\_processor.py`.

Q19: How does the system detect hand waving?

Answer: It tracks horizontal displacement `|Middle\_Finger\_Tip.x - Wrist.x| > 0.15` and vertical displacement `< 0.25` across frames over a sliding 2.0-second time window.

Project Code Context: Implemented in `detect\_wave()` in `live\_camera\_processor.py`.

Q20: How does MobileNetV2 classify gender?

Answer: The face/torso region (upper 60% of person box) is cropped, resized to `224x224`, normalized into a float tensor, and passed to MobileNetV2. Softmax outputs male vs female probability.

Project Code Context: Implemented in `classify\_gender()` in `live\_camera\_processor.py`.

Q21: What logic defines a 'Lone Woman at Night' emergency alert?

Answer: Four simultaneous conditions: ``is_nighttime() == True`` (19:00 to 06:00), ``Total_Persons == 1``, ``Gender == 'Female'``, and ``Gender_Confidence >= 0.60``.

Project Code Context: Evaluated inside ``process_live_camera()`` in ``live_camera_processor.py``.

Q22: Why did you set ``torch.no_grad()`` during MobileNetV2 inference?

Answer: Disabling autograd gradient tracking reduces CPU/GPU memory allocation and accelerates inference speed during model evaluation.

Project Code Context: See ``with torch.no_grad():`` in ``classify_gender()``.

Q23: How does frame skipping improve real-time performance?

Answer: Processing every frame slows down FPS. Setting ``FRAME_SKIP = 3`` skips inference on 2 out of 3 frames, maintaining smooth 30 FPS video rendering.

Project Code Context: See ``FRAME_SKIP = 3`` in ``live_camera_processor.py``.

Q24: What dataset is used for crime hotspot mapping?

Answer: ``women-crimedataset-India.csv``. It contains district-level Indian government records for 12 crime categories.

Project Code Context: Loaded into Pandas DataFrame ``df`` on ``app.py`` server startup.

Q25: How is total crime calculated if not present in the CSV?

Answer: Pandas sums columns 3 through 14 across rows: ``df['TOTAL_CRIMES'] = df[crime_columns].sum(axis=1)``.

Project Code Context: Executed during dataset loading in ``app.py``.

Q26: How does Geopy convert district names into coordinates?

Answer: Using ``Nominatim(user_agent='herwatch')``. The function ``geocode_with_retry()`` sends the location string to Nominatim API and receives latitude and longitude.

Project Code Context: Implemented in ``app.py`` lines 61-78.

Q27: What distance metric is used to find nearby hotspots?

Answer: Geodesic distance calculated via ``geodesic((user_lat, user_lon), (row['Latitude'], row['Longitude'])).kilometers``.

Project Code Context: Filters locations within ``MAX_DISTANCE_KM = 100`` in ``app.py``.

Q28: How are risk levels represented visually on the Folium map?

Answer: Red circle markers for High Risk (>1000 total crimes), Orange for Medium Risk, and Green for Low Risk.

Project Code Context: Rendered via ``folium.CircleMarker()`` in ``app.py``.

Q29: What protocol is used for live video streaming?

Answer: WebSockets (``ws://localhost:5001/ws/camera``) via ``flask-sock`` and native browser WebSocket API.

Project Code Context: Enables full-duplex, low-latency streaming without HTTP polling overhead.

Q30: How are annotated frames returned to the frontend?

Answer: OpenCV draws red boxes and alert text onto the frame matrix. The frame is encoded to JPEG Base64 and sent over the WebSocket as a JSON payload.

Project Code Context: Processed in `ws\_camera()` in `app.py`.

Q31: What happens if the WebSocket connection drops unexpectedly?

Answer: `LiveCamera.js` detects `ws.onclose`, sets streaming state to false, and displays an error alert prompting the user to reconnect.

Project Code Context: Handled in `startCamera()` error block in `LiveCamera.js`.

Q32: How does the frontend sound audio alerts?

Answer: When an alert is present in the received socket payload, `LiveCamera.js` invokes an HTML5 Audio object to play a warning alert beep.

Project Code Context: Implemented in `playAlertSound()` in `LiveCamera.js`.

Q33: How is user login currently structured in the codebase?

Answer: Login UI is built in `Login.js` with form state tracking. Form submission redirects to `/dashboard`. Authentication logic is client-side mocked.

Project Code Context: See `handleSubmit` in `Login.js`.

Q34: How does PrivateRoute protect frontend routes?

Answer: In `App.js`, `PrivateRoute` checks `isAuthenticated`. If true, it renders child components; if false, it redirects to `/login`.

Project Code Context: Currently mocked with `const isAuthenticated = true` in `App.js`.

Q35: How would you secure API endpoints with JWT tokens?

Answer: Implement `flask-jwt-extended`. Upon valid login at `/api/login`, return a signed JWT access token. Secure endpoints with `@jwt\_required()` decorator.

Project Code Context: Planned enhancement for production deployment.

Q36: What issue occurred with NumPy 2.x and how was it resolved?

Answer: PyTorch and OpenCV were compiled against NumPy 1.x ABI. Installing NumPy 2.2.6 caused a runtime crash (`Failed to initialize NumPy`). Resolved by installing `numpy==1.26.4` (`numpy<2`).

Project Code Context: Resolved in virtual environment (`venv`).

Q37: Why did `npm start` fail on macOS initially?

Answer: `package.json` contained Windows CMD syntax (`set NODE\_OPTIONS=... && react-scripts start`). Updated to standard cross-platform syntax (`NODE\_OPTIONS=... react-scripts start`).

Project Code Context: Fixed in `sos-detection-frontend/package.json`.

Q38: How do you prevent memory leaks when processing video feeds?

Answer: By limiting queue sizes (`queue.Queue(maxsize=10)`), releasing OpenCV capture objects (`cap.release()`), and closing WebSockets on component unmount.

Project Code Context: Implemented across `app.py` and `LiveCamera.js`.

Q39: What happens if Geopy Nominatim times out?

Answer: `geocode\_with\_retry()` catches `GeocoderTimedOut` and retries up to 3 times with exponential backoff delays.

Project Code Context: Implemented in `app.py` line 61.

Q40: How do you verify backend API health before rendering UI components?

Answer: `HotspotMap.js` issues a `fetch('/api/health')` request in `useEffect()`. If status is 200, it enables search inputs; otherwise it displays a backend error alert.

Project Code Context: See `checkApiStatus()` in `HotspotMap.js`.

Q41: Why use YOLOv8 Nano instead of YOLOv8 Large?

Answer: YOLOv8 Nano has only 3.2 million parameters, offering inference speeds over 60 FPS on CPU, which is crucial for real-time video processing.

Project Code Context: Model loaded via `YOLO('yolov8n.pt')`.

Q42: What is the purpose of `cv2.cvtColor(img, cv2.COLOR\_RGB2BGR)`?

Answer: PIL and browser canvas images use RGB color order, whereas OpenCV processes images in BGR format. Conversion is required for correct color representation.

Project Code Context: See `base64\_to\_cv2()` in `app.py`.

Q43: What is the role of `queue.Queue(maxsize=10)` in `app.py`?

Answer: It buffers incoming WebSocket frames to prevent memory exhaustion if model inference takes slightly longer than frame capture rate.

Project Code Context: Global variable in `app.py`.

Q44: How does `useRef` differ from `useState` in `LiveCamera.js`?

Answer: `useState` causes component re-renders when updated. `useRef` holds mutable references (like the WebSocket instance `wsRef`) without causing unnecessary re-renders.

Project Code Context: See `const wsRef = useRef(null)` in `LiveCamera.js`.

Q45: How would you scale this application for 10,000 concurrent cameras?

Answer: Offload video stream ingestion to WebRTC / RTSP media servers, deploy Flask backend across Kubernetes pods with GPU acceleration, and use Redis for WebSocket message pub/sub.

Project Code Context: Recommended scaling architecture.

Q46: Why is dataset loading performed inside a `try-except` block at startup?

Answer: To prevent server crash if `women-crimedataset-India.csv` is missing or corrupted. If loading fails, `df` is set to `None` and relevant endpoints return HTTP 500 error messages.

Project Code Context: See lines 35-43 in `app.py`.

Q47: How does `React.memo` or component optimization help in this dashboard?

Answer: Prevents unnecessary re-renders of static UI cards and chart canvases when video frame state updates rapidly.

Project Code Context: Performance optimization technique.

Q48: What is the benefit of serving Folium maps via iframe in React?

Answer: Folium outputs raw HTML/JavaScript leaflet code. Embedding via `` isolates external Leaflet scripts and styles from conflicting with React DOM.

Project Code Context: Implemented in `HotspotMap.js`.

Q49: What metric determines if a gesture counts as SOS hand waving?

Answer: Horizontal wrist-to-finger tip displacement crossing the `0.15` threshold at least `MIN\_WAVE\_COUNT = 2` times within a 2-second interval.

Project Code Context: Evaluated in `detect\_sos\_gesture()`.

Q50: How do you explain your personal contribution to this project?

Answer: "I worked on integrating the multi-model AI pipeline (YOLOv8, MediaPipe, MobileNetV2) into a unified Flask WebSocket backend, fixing environment compatibility issues, and connecting real-time telemetry to the React dashboard UI."

Project Code Context: Accurate, professional interview answer.

14. "Explain This Code" — Crucial Code Snippets

Snippet 1: Base64 to OpenCV BGR Conversion (ThemeBased Code/app.py)

```
def base64_to_cv2(base64_string):
    img_data = base64.b64decode(base64_string.split(',')[1])
    img = Image.open(io.BytesIO(img_data))
    return cv2.cvtColor(np.array(img), cv2.COLOR_RGB2BGR)
```

Explanation: Splits the Data URL header, decodes Base64 bytes into PIL Image, converts PIL matrix to NumPy array, and shifts color space from RGB to OpenCV BGR format.

Snippet 2: MobileNetV2 Gender Inference (ThemeBased Code/live_camera_processor.py)

```
face_img = cv2.resize(face_img, (224, 224))
face_tensor = torch.tensor(face_img, dtype=torch.float32).permute(2, 0, 1).unsqueeze(0) / 255.0
with torch.no_grad():
    output = gender_model(face_tensor)
    pred_idx = output.argmax().item()
    confidence = torch.softmax(output, dim=1)[0][pred_idx].item()
```

Explanation: Resizes crop to 224x224, converts HWC image to CHW PyTorch tensor, normalizes pixel values to [0,1], disables gradient tracking, passes tensor into MobileNetV2, and computes Softmax confidence.

Snippet 3: Geodesic Crime Hotspot Calculation (ThemeBased Code/app.py)

```
df_copy['distance'] = df_copy.apply(
    lambda row: geodesic((user_lat, user_lon), (row['Latitude'], row['Longitude'])).kilometers
    if not pd.isna(row['Latitude']) else float('inf'), axis=1
)
nearby_hotspots = df_copy[df_copy['distance'] <= 100].sort_values(by='distance')
```

Explanation: Applies Geopy geodesic spherical formula row-by-row against user coordinates, filters locations within 100km, and sorts hotspots in ascending order of proximity.

Snippet 4: WebSocket Real-Time Frame Handler (ThemeBased Code/app.py)

```
@sock.route('/ws/camera')
def ws_camera(ws):
    while True:
        data = ws.receive()
        frame_data = json.loads(data).get('frame')
        detections = process_frame(frame_data)
        ws.send(json.dumps(detections))
```

Explanation: Establishes persistent full-duplex WebSocket connection. Continuously receives JSON frame payloads, processes detections, and sends annotated Base64 response back to React client.

15. Real Debugging Case Studies & Fixes

Case Study 1: NumPy 2.x PyTorch C-ABI Crash

- **Problem:** Starting Flask server (`app.py`) produced `UserWarning: Failed to initialize NumPy: \_ARRAY\_API not found` followed by PyTorch C-extension initialization crash.
- **Root Cause:** PyTorch 2.2.2 and OpenCV 4.13 installed in `venv` were compiled against NumPy 1.x C-ABI specifications. NumPy automatically upgraded to version 2.2.6, breaking backwards compatibility.
- **Resolution:** Downgraded virtual environment NumPy package to version 1.26.4 (`pip install 'numpy<2'`). Verified backend boot success.

Case Study 2: Windows `set` Command Failure on macOS

- **Problem:** Running `npm start` in `sos-detection-frontend` failed on macOS zsh shell.
- **Root Cause:** `package.json` contained Windows CMD syntax: `"start": "set NODE\_OPTIONS=... && react-scripts start"`.
- **Resolution:** Modified `package.json` scripts to standard cross-platform format: `"start": "NODE\_OPTIONS=--openssl-legacy-provider react-scripts start"`.

16. Ultimate 5-Minute Pre-Interview Cheat Sheet

■ HerWatch in 5 Lines:

1. HerWatch is a real-time AI women's safety surveillance system built with React and Flask.
2. Uses YOLOv8 for person detection, MediaPipe for SOS gesture recognition, and MobileNetV2 for gender classification.
3. Triggers emergency alerts for non-verbal distress signals and solitary women at night (19:00 - 06:00).
4. Integrates Geopy and Folium to plot interactive 100km crime hotspot heatmaps from Indian crime datasets.
5. Streams live video frames over WebSockets (`/ws/camera`) to deliver sub-second dashboard alerts.

■ Tech Stack in 5 Lines:

- Frontend: React 18, Material-UI, Framer Motion, Chart.js
- Backend: Flask 3.1, Flask-CORS, Flask-Sock (WebSockets)
- Computer Vision: YOLOv8n, MediaPipe Holistic, MobileNetV2 (PyTorch)
- Data & Geospatial: Pandas, Geopy (Nominatim & Geodesic), Folium Leaflet
- Ports: Frontend = 3000, Backend = 5001

■ Top 5 Commands to Remember:

1. Backend Run: `cd "ThemeBased Code" && source venv/bin/activate && python app.py`
2. Frontend Run: `cd "ThemeBased Code/sos-detection-frontend" && npm start`
3. Health Check: `curl http://localhost:5001/api/health`
4. WebSocket URL: `ws://localhost:5001/ws/camera`
5. Crime Dataset File: `women-crimedataset-India.csv`

End of Complete Project Interview Preparation Guide — HerWatch Project.